

Reviewer Pack — Sheaf Cohomology Euler Characteristic and Tseitin Resolution...

Entry 62b0c1d5061d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 02:31:19 UTC

Sheaf Cohomology Euler Characteristic and Tseitin Resolution Proof Size

Entry ID: 62b0c1d5061d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 02:31:19 UTC

1. Conjecture

Field A (mathematical branch): Sheaf Cohomology **Field B** (complexity object): Resolution Proof Size for Tseitin Formulas

Statement:

For a Tseitin formula derived from a graph G with n vertices, the absolute value of the Euler characteristic of the sheaf cohomology groups $H^i(G)$ is inversely proportional to the resolution proof size, i.e., $|\chi(H^i(G))| = \Theta(1 / \log n)$.

Rationale (proposer's reasoning):

Sheaf cohomology captures the topological structure of the graph, which may influence the complexity of the resolution proof. The Euler characteristic provides a quantitative measure of this topological invariant, potentially linking it to the proof size.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2b824f3b3876e34d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 20 # Number of vertices in the graph
    if n < 5 or n > 40:
        return {
            "metric_name": "resolution_proof_size",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "n_out_of_range"
        }

    # Generate a random graph
    edges = set()
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < 0.5:
                edges.add((i, j))

    # Create the Tseitin formula
    clauses = []
    literals = [f"v{i+1}" for i in range(n)]
    for (i, j) in edges:
        clauses.append([literals[i], literals[j]])
        clauses.append([-literals[i], -literals[j]])
        clauses.append([literals[i], -literals[j]])
        clauses.append([-literals[i], literals[j]])

    # Add the negation of each literal
    for i in range(n):
        clauses.append([-literals[i]])

    # DPLL-based resolution proof size estimation (simplified)
    def dpll(clauses, assignment):
        if not clauses:
            return 0
```

```

unit_clauses = [c[0] for c in clauses if len(c) == 1]
pure_literals = {}
for c in clauses:
    for l in c:
        if l.startswith('v'):
            if l not in pure_literals:
                pure_literals[l] = True
            elif not pure_literals[l]:
                pure_literals[l] = False
        else:
            if -l not in pure_literals:
                pure_literals[-l] = True
            elif not pure_literals[-l]:
                pure_literals[-l] = False

unit_clauses.extend([p for p, v in pure_literals.items() if v])
unit_clauses.extend([-p for p, v in pure_literals.items() if not v])

if not unit_clauses:
    return 1 + max(dpll(clause, assignment) for clause in clauses)

literal = unit_clauses[0]
new_assignment = assignment.copy()
new_assignment[literal] = True
true_clauses = [c for c in clauses if literal not in c and -literal not in c]
false_clauses = [c for c in clauses if literal in c or -literal in c]

if any(l.startswith('v') and l not in new_assignment and -l not in new_assignment for l in true_clauses):
    return 1 + dpll(true_clauses, new_assignment)

new_assignment[literal] = False
true_clauses = [c for c in clauses if literal not in c and -literal not in c]
false_clauses = [c for c in clauses if literal in c or -literal in c]

if any(l.startswith('v') and l not in new_assignment and -l not in new_assignment for l in false_clauses):
    return 1 + dpll(false_clauses, new_assignment)

return 1

proof_size = dpll(clauses, {})

# Euler characteristic of the sheaf cohomology groups (constant sheaf)
euler_characteristic = n - len(edges) + 1

return {
    "metric_name": "resolution_proof_size",
    "metric_value": abs(euler_characteristic),
    "instances_tested": 1,
    "conjecture_holds": proof_size > 0 and abs(euler_characteristic) / math.log(n) < 100,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = "mapping_undefined"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ff24d2c.py", line 113, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ff24d2c.py", line 43, in run_trial
    clauses.append([-literals[i], -literals[j]])
                    ~~~~~^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in clause generation, preventing data collection | next: Fix the TypeError by ensuring literals are integers before applying unary negation

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136382
2	propose	ollama_remote	qwen3:8b	0	0	168978
3	preregistration	ollama_remote	qwen3:8b	0	0	31593
4	novelty	ollama_remote	qwen3:8b	0	0	27521
5	novelty	ollama_remote	qwen3:8b	0	0	27060
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14178
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9915
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13471
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13251
10	judge	ollama_remote	qwen3:8b	0	0	22885

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 465235 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/62b0c1d5061d.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/62b0c1d5061d.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/62b0c1d5061d.tar.gz* (if generated)